

APIOBike

APIOBike APIO City'de yeni kurulmuş bir bisiklet paylaşım servsidir. Şehir boyunca bisiklet ödünç almaya veya teslim etmeye imkan tanıyan birkaç istasyon kurulmuştur. Uygunluğu ve düşük maliyetinden dolayı, hızlıca APIO City'deki en önemli ulaşım yoluna dönüşmüştür. Ancak, APIOBike diğer bisiklet paylaşım servislerinin de karşılaştığı ortak bir problemle karşılaştı: farklı istasyonlar arasındaki ödünç alma ve teslim etme oranı dengesizliği. Bu, bazı istasyonlarda çok az bisiklet varken, yakınlarda yaşayanları ödünç almaktan mahrum ederken; diğer istasyonlarda, yaşayanların ihtiyaç duymadığı bisikletlerin fazla depolanmasına neden olur.

Bu sorunu çözmek için, APIOBike her istasyonda uygun sayıda bisiklet olmasını sağlayacak şekilde, her akşam istasyonlar arasında bisikletleri yeniden dağıtan bir yeniden dengeleme kamyonu sevk etmeyi planlıyor. APIO City'de $0, 1, \dots, N - 1$ şeklinde numaralanmış toplamda N istasyon var. APIOBike, her gece i istasyonunda her zaman sabit $A[i]$ sayıda bisiklet olduğunu, ve kimsenin gece bisiklet ödünç almadığını ve teslim etmediğini gözlemlemiştir. Şirket her sabah i istasyonunda tam olarak $B[i]$ bisiklet olmasını hedefliyor.

N istasyon arasında, yeniden dengeleme kamyonu için $N - 1$ tane belirlenmiş yol bulunmaktadır. Her yol iki farklı istasyonu bağlıyor ve kamyon sadece bu yollarda hareket edecek şekilde sınırlanmıştır. Her yol bir birim uzunluktadır. İstasyonlar ağı, kamyonun herhangi istasyon ikilisi arasında dolaşabileceğini sağlayacak şekilde bağlıdır (connected). Her gece, APIOBike i istasyonunda tam olarak $B[i]$ bisiklet olmayı sağlamak için yeniden dağıtım kamyonunu sevk edecektir. Kamyon rotasına herhangi bir istasyonda başlayabilir ve rotayı herhangi bir istasyonda bitirebilir.

Kamyon yeniden dengelemenin başında boştur. Kamyon istasyondayken, sürücü herhangi sayıda bisikleti istasyondan kamyonu yükleyebilir veya kamyonun istasyona herhangi sayıda bisikleti bırakabilir. Kamyon ve istasyonların bisikletler için sınırsız kapasitesi vardır ama herhangi bir konumdaki bisikletlerin sayısı sıfırın altına düşemez. Şirket, yeniden dağıtım için olası en küçük dolaşım mesafesini ve buna karşılık gelen stratejiyi belirlemek istiyor.

Kodlama Detayları

Aşağıdaki prosedürü kodlamalısınız.

```
std::pair<std::vector<int>, std::vector<long long>>
find_rebalancing_strategy(int N,
                           std::vector<int> A,
                           std::vector<int> B,
                           std::vector<int> U,
                           std::vector<int> V)
```

- A : N uzunluğunda bir dizi, $A[i]$ her gece i istasyonundaki bisiklet sayısını verir.
- B : N uzunluğunda bir dizi, $B[i]$ her gece i istasyonundaki hedef bisiklet sayısını gösterir.
- U, V : $N - 1$ uzunluğunda yol ağını temsil eden diziler. Her $0 \leq i < N - 1$ için, i -inci yol istasyon $U[i]$ ve istasyon $V[i]$ 'yi bağlar.
- Bu prosedür her test durumu için **en fazla 150 000 kez** çağırılır.

Prosedür bir dizi ikilisi döndürmelidir (X, Y) . Diziler eşit ve $k + 1$ uzunluğundadır ve yeniden dengeleme stratejisini temsil eder:

- X : Ziyaret edilen istasyon indislerinin kronolojik sırada oluşturduğu dizi.
- Y : Ziyaret edilen her istasyondaki net bisiklet teslimatı. Her $0 \leq j \leq k$ için:
 - Eğer $Y[j] \geq 0$ ise, sürücü $Y[j]$ bisikleti $X[j]$ istasyonuna kamyonla bırakır, istasyondaki bisiklet sayısı $Y[j]$ kadar artar.
 - Eğer $Y[j] < 0$ ise, sürücü $-Y[j]$ bisikleti $X[j]$ istasyonundan kamyonla alır, istasyondaki bisiklet sayısı $-Y[j]$ kadar azalır.

valid bir k dolaşım mesafeli yeniden dengeleme stratejisi (X, Y) aşağıdaki koşulları sağlamalıdır:

- Her $0 \leq j \leq k$ için, $0 \leq X[j] < N$.
- Her $0 \leq j < k$ için, istasyonlar $X[j]$ ve $X[j + 1]$ bir yol ile doğrudan bağlıdır.
- Her $0 \leq j \leq k$ için, $\sum_{t=0}^j Y[t] \leq 0$.
- Her istasyon $0 \leq i < N$ için ve her adım $0 \leq j \leq k$ için, diyelim ki $S_{i,j}$ bütün t adımlarındaki $Y[t]$ 'ler toplamı olsun öyle ki $0 \leq t \leq j$ ve $X[t] = i$.
 - i istasyonundaki bisiklet sayısı hiçbir zaman sıfırın altına düşmez: $A[i] + S_{i,j} \geq 0$.
 - Strateji sona erdiğinde, her istasyon i kesinlikle $B[i]$ bisiklet içermelidir: $A[i] + S_{i,k} = B[i]$.

Sınırlar

- $2 \leq N \leq 300\,000$
- Bütün `find_rebalancing_strategy` çağrılarındaki N değerleri toplamı her testte 300 000 değerini geçmez.
- $0 \leq U[i], V[i] < N$ ve $U[i] \neq V[i]$ her i için öyle ki $0 \leq i < N$.
- $0 \leq A[i], B[i] \leq 10^9$ her i için öyle ki $0 \leq i < N$.

- Herhangi istasyon ikilisi arasında dolaşım mümkündür.
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$.
- En azından bir i bulunur öyle ki $0 \leq i < N$ ve $A[i] \neq B[i]$.

Subtasklar ve Skorlama

Burada, T `find_rebalancing_strategy` çağrılarının sayısıdır, ve $\sum N$ bütün `find_rebalancing_strategy` çağrılarındaki N değerlerinin toplamıdır.

Subtask	Skor	Ek Sınır
1	4	İstasyonlar ve yollar P özelliğini sağlar (aşağı bakınız) Tam olarak bir tane i vardır öyle ki $0 \leq i < N$ ve $A[i] > 0$.
2	11	$T \leq 10$. $N \leq 7$. İstasyonlar ve yollar P özelliğini sağlar.
3	16	İstasyonlar ve yollar P özelliğini sağlar.
4	9	Tam olarak bir tane i vardır öyle ki $0 \leq i < N$ ve $A[i] > 0$.
5	24	$\sum N \leq 500$.
6	15	$\sum N \leq 5000$.
7	21	Hiçbir ek kısıt yoktur.

Özellik P: Her $0 \leq i < N - 1$ için, $U[i] = i$ ve $V[i] = i + 1$. Her $0 \leq i < N$ için, $A[i] \neq B[i]$.

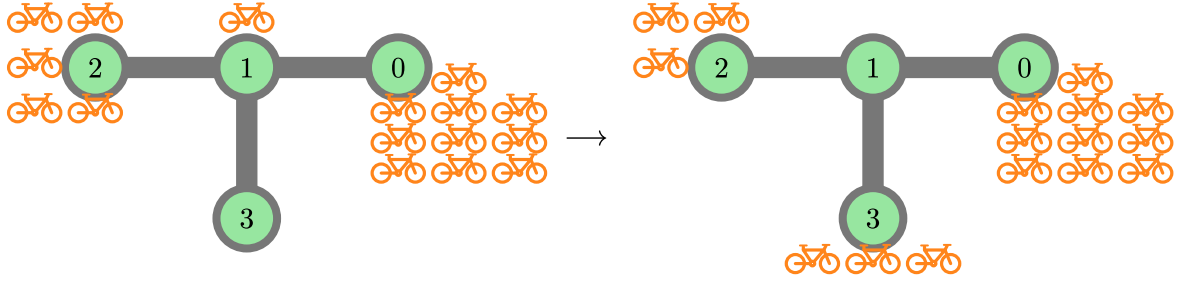
Eğer döndürülen diziler X ve Y tam olarak $k^* + 1$ uzunluğa sahipse, burada k^* en küçük dolaşım mesafesini temsil eder, ama (X, Y) valid bir strateji değilse, skorun 50%'lik kısmını alırsınız.

Örnek

Örnek 1

Aşağıdaki çağrıyı düşününüz:

```
find_rebalancing_strategy(4,
                        [10, 1, 5, 0],
                        [10, 0, 3, 3],
                        [0, 1, 1],
                        [1, 2, 3])
```



APIO City'de $N = 4$ istasyon vardır. Başlangıçta, istasyonlar $A = [10, 1, 5, 0]$ bisiklete sahiptir, ve hedef her istasyonda $B = [10, 0, 3, 3]$ bisiklete sahip olmaktır.

Varsayalım ki yeniden dengeleme kamyonu 2 istasyonundan başlasın, ve şu adımları takip etsin:

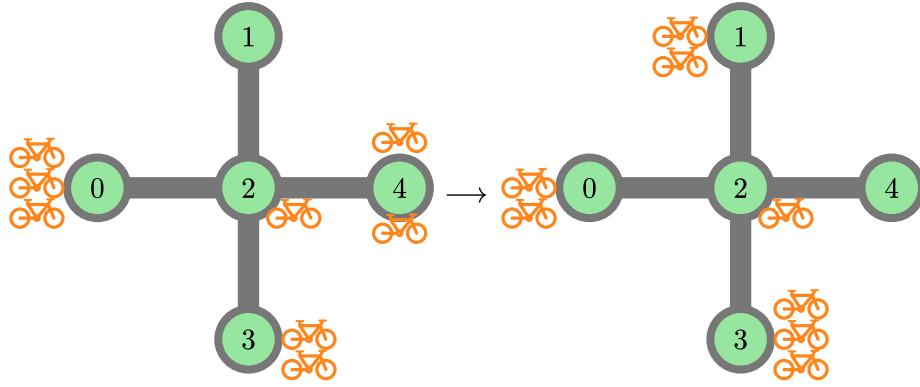
- 2 istasyonunda, 2 bisikleti kamyonu yükle. 2 istasyonu şuanda 3 bisiklete sahip olur, ve kamyonunda 2 bisiklet olur.
- 1 istasyonuna hareket et ve 1 bisikleti kamyonu yükle. 1 istasyonu şuanda 0 bisiklete sahip, ve kamyonunda 3 bisiklet var.
- 3 istasyonuna hareket et, ve 3 bisiklet bırak. 3 istasyonu şuanda 3 bisiklete sahip, ve kamyonunda 0 bisiklet var.

Toplam dolaşım mesafesi 2 olur. Bu strateji için ilgili döndürülecek diziler $X = [2, 1, 3]$ ve $Y = [-2, -1, 3]$ olur. Bu stratejinin en küçük mesafeye sahip olduğu ispatlanabilir. Sonuçta, prosedür $([2, 1, 3], [-2, -1, 3])$ dönmelidir.

Örnek 2

Aşağıdaki çağırıcıyı düşününüz:

```
find_rebalancing_strategy(5,
                        [3, 0, 1, 2, 2],
                        [2, 2, 1, 3, 0],
                        [2, 2, 2, 2],
                        [0, 4, 3, 1])
```



APIO City'de $N = 5$ istasyon vardır. Başlangıçta, istasyonlarda $A = [3, 0, 1, 2, 2]$ bisiklet vardır, ve hedef her istasyonda $B = [2, 2, 1, 3, 0]$ bisiklete sahip olmaktır.

Varsayalım ki yeniden dengeleme kamyonu 0 istasyonundan başlasın, ve şu adımları izlesin:

- 0 istasyonunda, 1 bisiklet yükle.
- 2 istasyonuna hareket et ve 1 bisiklet yükle.
- 1 istasyonuna hareket et ve 2 bisikleti bırak.
- 2 istasyonuna hareket et.
- 4 istasyonuna hareket et ve 2 bisiklet yükle.
- 2 istasyonuna hareket et ve 1 bisiklet bırak.
- 3 istasyonuna hareket et ve 1 bisiklet bırak.

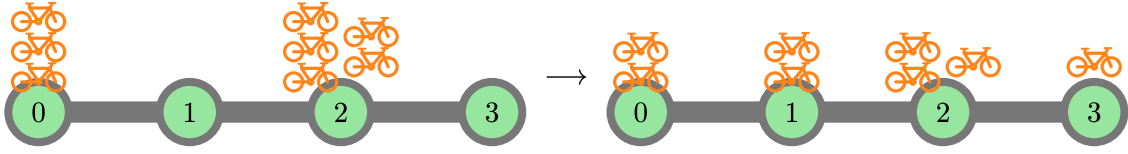
Toplam mesafe 6 olur. Bu strateji için ilgili döndürülmesi gereken diziler $X = [0, 2, 1, 2, 4, 2, 3]$ ve $Y = [-1, -1, 2, 0, -2, 1, 1]$ olur. Bu stratejinin en küçük mesafeyi verdiği ispatlanabilir. Sonuç olarak, prosedür $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$ döndürmelidir.

6 mesafeli başka stratejiler de, örneğin $X = [0, 2, 3, 2, 4, 2, 1]$ ve $Y = [-1, 0, 1, 0, -2, 0, 2]$, doğru kabul edilecektir. Dönüş dizileri X, Y $7 = 6 + 1$ uzunluğunda ise ama doğru stratejiye sahip değilse, örneğin $X = Y = [0, 0, 0, 0, 0, 0, 0]$, skorun 50%'si alınır.

Örnek 3

Aşağıdaki çağrıyı ele alınız:

```
find_rebalancing_strategy(4,
                        [3, 0, 5, 0],
                        [2, 2, 3, 1],
                        [0, 1, 2],
                        [1, 2, 3])
```



Optimal çözüm $X = [2, 1, 0, 1, 2, 3]$ ve $Y = [-1, 1, -1, 1, -1, 1]$ olur. Prosedür $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$ döndürmelidir. Diğer valid stratejiler de, örneğin $X = [2, 1, 0, 1, 2, 3]$ ve $Y = [-2, 2, -1, 0, 0, 1]$, doğru kabul edilecektir.

Bu örnek subtask 2 ve 3 kısıtlarını sağlar.

Örnek Grader

Girdinin ilk satırı sadece bir tam sayı T içerir, seneryoların sayısı. T seneryonun tanımı şu şekildedir, her birisi aşağıdaki formattadır.

Girdi formatı:

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

Çıktı Formatı:

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```